Week 4 – Thread & Concurrency 1

Operating Systems Ch 4 (Sections 4.1 – 4.4)

Korea University Sejong Campus, Department of Computer Science & Software

Table of Contents

0. **Quiz** – Beginning of 1st Hour
1. Overview: Concept and Structure of Threads
2. Motivation for Multithreading
3. Multicore Programming & Amdahl's Law
4. Data Parallelism vs Task Parallelism
5. Multithreading Models
6. Thread Libraries (Pthreads, Windows, Java)
7. Java Executor Framework
8. Lab – Pthreads Multithreaded Programming
9. Summary and Next Week Preview

Part 1

Overview: The Concept of Threads

Silberschatz Ch 4.1

What Is a Thread?

Thread = a **unit of CPU utilization** within a process

What each thread **independently owns**:

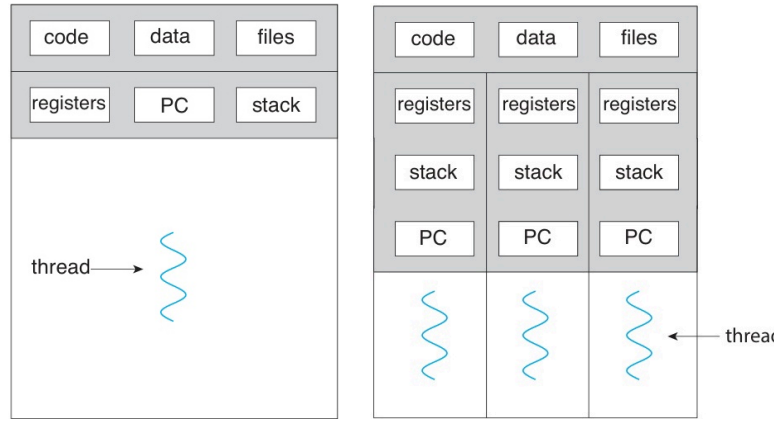
- Thread ID
- Program Counter (PC)
- Register Set
- **Stack** (function calls, local variables)

What threads in the same process **share**:

- Code section (program code)
- Data section (global variables)
- OS resources (open files, signals, etc.)

> Traditional process = single thread of control Modern process = multiple threads of control

Single-Threaded vs Multithreaded Process (Figure 4.1)



Silberschatz, Figure 4.1 – Single-threaded and multithreaded processes

- Single-threaded: one execution flow, one PC, one stack
- Multithreaded: **multiple execution flows**, each with its own independent PC and stack
- **Code, data, and files are shared by all threads**

Thread vs Process Comparison

Category	Process	Thread
Creation cost	**High** (memory and resource allocation)	**Low** (only stack and registers needed)
Context Switch	**Slow** (address space switch)	**Fast** (same address space)
Memory sharing	Isolated by default (IPC required)	**Naturally shared** (code, data)
Independence	High (one dying has little effect on others)	Low (one dying can affect the entire process)
Communication	IPC required (pipe, socket, etc.)	Direct communication via global variables

> Threads are also called **"lightweight processes."**

Why Use Threads? — Real-World Examples

Web Browser:

- Thread 1: Receive data from the network
- Thread 2: Render images and text
- Thread 3: Handle user input

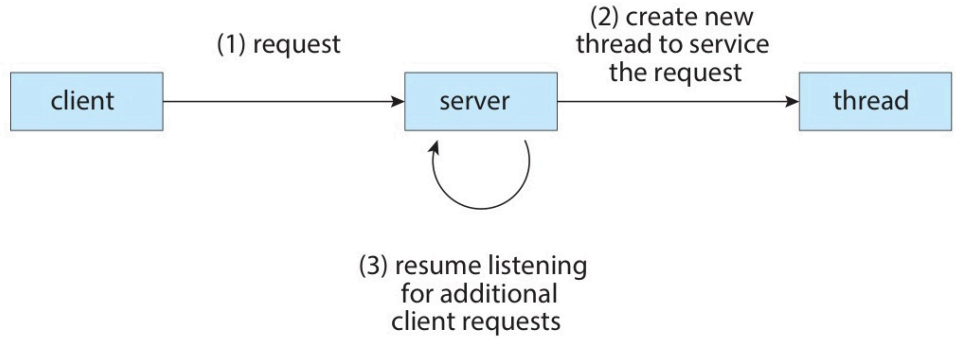
Word Processor:

- Thread 1: Display the document on screen
- Thread 2: Handle keystrokes
- Thread 3: Spell/grammar checking (background)

Photo App — Thumbnail Generation:

- Each image in a collection is processed by a separate thread
- Parallel processing reduces total elapsed time

Multithreaded Web Server (Figure 4.2)



Silberschatz, Figure 4.2 – Multithreaded server architecture

Traditional approach: Create a new **process** per request → time-consuming, resource-wasteful

Modern approach: Create a new **thread** per request

- Shares the same address space, so it is **fast and lightweight**
- Advantageous for handling large numbers of concurrent connections

The Linux kernel is also multithreaded – `kthreadd` (pid=2) is the parent of all kernel threads

Part 2

Benefits of Multithreading

Silberschatz Ch 4.1.2

Four Benefits of Threads

1. Responsiveness

- The UI thread can continue to respond
- Long-running operations execute **asynchronously** in a separate thread
- Example: the interface remains responsive even during lengthy processing after a button click

2. Resource Sharing

- Sharing between processes requires shared memory or message passing
- Threads share code, data, and files **by default**
- Multiple activities can be performed within the same address space

Four Benefits of Threads (continued)

3. Economy

- Thread creation is **much cheaper** than process creation
  - Lower memory allocation and resource allocation costs
- Context switching between threads is also **much faster**
  - Same address space, so no TLB flush is needed

4. Scalability

- True **parallel execution** is possible on multiprocessor/multicore systems
- A single-threaded process uses **only 1 core** no matter how many are available
- A multithreaded process can run each thread on a different core simultaneously

Thread Benefits Summary Table

Benefit	Key Explanation	Representative Example
Responsiveness	Program remains responsive during blocking	Web browser UI
Resource Sharing	Automatic resource sharing within the same address space	Global variable access
Economy	Creation/switching costs are lower than processes	Web server request handling
Scalability	Parallel execution on multicore systems	Video encoding, scientific computation

> If a web server creates threads instead of processes per request, overhead is significantly reduced.

Part 3

Multicore Programming

Silberschatz Ch 4.2

Concurrency vs Parallelism

Concurrency

- Multiple tasks **making progress** at the same time
- Possible even on a single core – **interleaving** via time-sharing

```
Single core:  T1  T2  T3  T4  T1  T2  T3  T4  T1 ...
              ──────────────────────────────────▷ time
```

Parallelism

- Multiple tasks **actually executing simultaneously**
- Only possible on multi-core – each core runs a different thread

```
Core 1:  T1      T3      T1      T3      T1 ...
Core 2:  T2      T4      T2      T4      T2 ...
         ──────────────────────────────────▷ time
```

> Concurrency without parallelism is possible, but parallelism without concurrency is not

Single-Core vs Multi-Core Execution Comparison

Single-core system (Figure 4.3):

- 4 threads: T1, T2, T3, T4
- Only one runs at a time → concurrency achieved through **interleaving**
- Not true parallelism

Multi-core system (Figure 4.4):

- Core 1: alternates between T1 and T3
- Core 2: alternates between T2 and T4
- T1 and T2 run **simultaneously** → **true parallelism**

Category	Single-core	Multi-core
Concurrency	O (interleaving)	O (parallel)
Parallelism	X	**O**

Five Challenges of Multicore Programming

Challenges that must be overcome to **effectively utilize** multicore systems:

1. Identifying tasks

- Find separable **independent** tasks
- Ideally, there should be no dependencies between tasks

2. Balance

- Distribute **equal workloads** to each core
- Assigning a separate core to a low-contribution task is inefficient

3. Data splitting

- **Split data appropriately** alongside tasks and distribute to each core
- Example: divide an array into N equal parts, each core handles one part

Five Challenges of Multicore Programming (continued)

4. Data dependency

- If there are data dependencies between tasks, **synchronization** is required
- If Task B needs the result of Task A, their order must be guaranteed
- Covered in detail in Ch 6

5. Testing and debugging

- **Various execution paths** are possible during parallel execution
- **Non-deterministic** results can occur
- Bugs that are hard to reproduce (heisenbugs)

> Due to these challenges, many experts argue that the multicore era requires a **fundamentally new approach** to software design.

Amdahl's Law

A law that expresses the limit of overall performance improvement when only **part of a system** is improved

$$\text{speedup} \leq \frac{1}{S + \frac{1-S}{N}}$$

- **S** = serial portion (fraction of serial execution)
- **N** = number of processing cores

Key meaning:

- Even as N approaches infinity, speedup converges to **1/S**
- The serial portion acts as the **bottleneck**

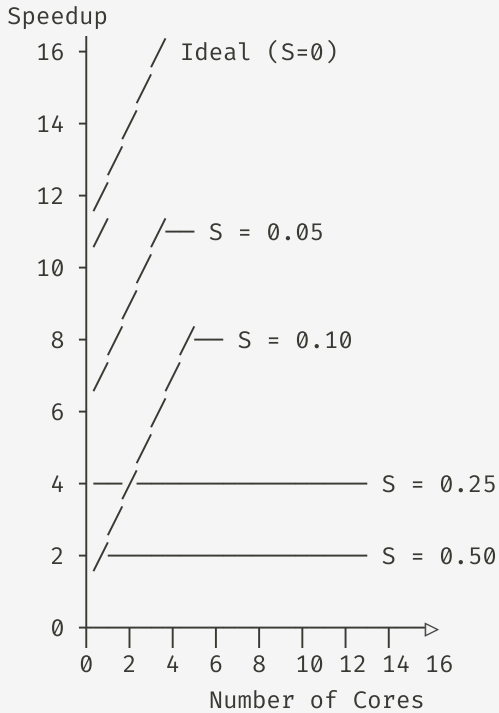
Amdahl's Law – Numerical Examples

S (serial fraction)	N = 2	N = 4	N = 8	N → ∞
5%	1.90x	3.48x	5.93x	**20.00x**
10%	1.82x	3.08x	4.71x	**10.00x**
25%	1.60x	2.28x	3.02x	**4.00x**
50%	1.33x	1.60x	1.78x	**2.00x**

Example:

- A program that is 75% parallel + 25% serial
  - 2 cores → **1.6x** speedup
  - 4 cores → **2.28x** speedup

Amdahl's Law – Graph



- The smaller S is (the higher the parallelization ratio), the greater the effect of adding cores
- If S = 0.50, the maximum is 2x regardless of the number of cores

Types of Parallelism

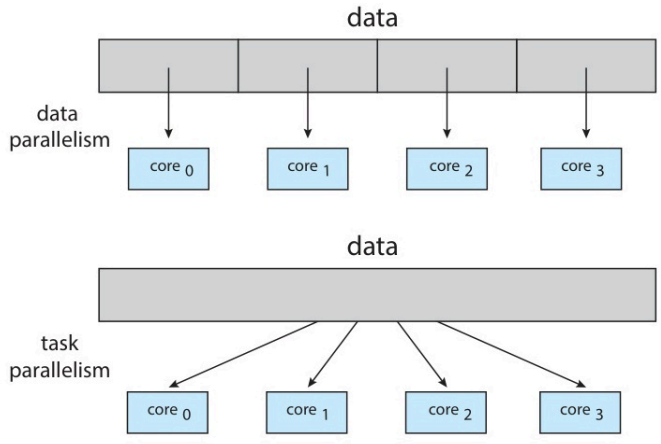
Data Parallelism

- Distribute the **same operation** across **subsets** of data
- Split the data; each core performs the same task
- Example: summing an array of size N
  - Core 0: sum elements [0] to [N/2-1]
  - Core 1: sum elements [N/2] to [N-1]

Task Parallelism

- Distribute **different operations (tasks)** to each thread
- Different tasks may operate on the same data
- Example: for an array
  - Core 0: compute the mean
  - Core 1: compute the standard deviation

Data Parallelism vs Task Parallelism Comparison



Silberschatz, Figure 4.5 – Data parallelism vs task parallelism

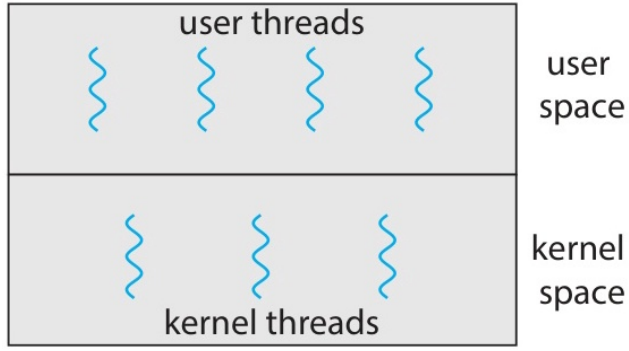
Category	Data Parallelism	Task Parallelism
Data distribution	**Partitioned** and distributed	Same data possible
Operation type	**Identical** operations	**Different** operations

Part 4

User Threads and Kernel Threads

Silberschatz Ch 4.3

User Threads vs Kernel Threads



Silberschatz, Figure 4.6 – User and kernel threads

User Threads:

- Managed by a user-level library (without kernel support)
- Examples: POSIX Pthreads, Windows threads, Java threads

Kernel Threads:

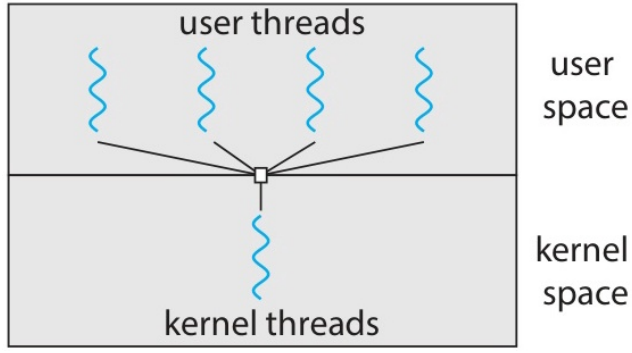
- **Directly managed** by the OS kernel
- Supported by virtually all modern OSes: Windows, Linux, macOS

Part 5

Multithreading Models

Silberschatz Ch 4.3.1 – 4.3.3

Many-to-One Model (Figure 4.7)



Silberschatz, Figure 4.7 – Many-to-one model

Multiple user threads mapped to **one** kernel thread

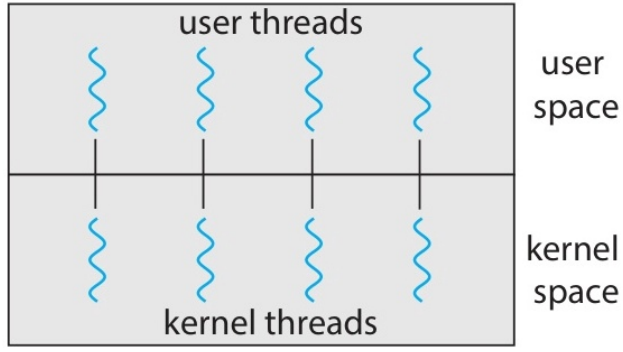
Advantages:

- Thread management is done in user space, making it **efficient**

Disadvantages:

- If one thread makes a **blocking system call → entire process blocks**
- **True parallelism is impossible** – the kernel can only schedule one thread at a time

One-to-One Model (Figure 4.8)



Silberschatz, Figure 4.8 – One-to-one model

Each user thread mapped to **one** kernel thread

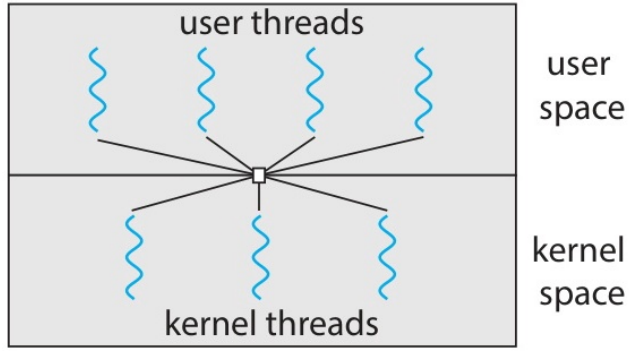
Advantages:

- Even if one thread blocks, **other threads can continue running**
- **True parallelism** achieved on multiprocessors

Disadvantages:

- Creating a user thread also **creates a kernel thread** → overhead

Many-to-Many Model (Figure 4.9)



Silberschatz, Figure 4.9 – Many-to-many model

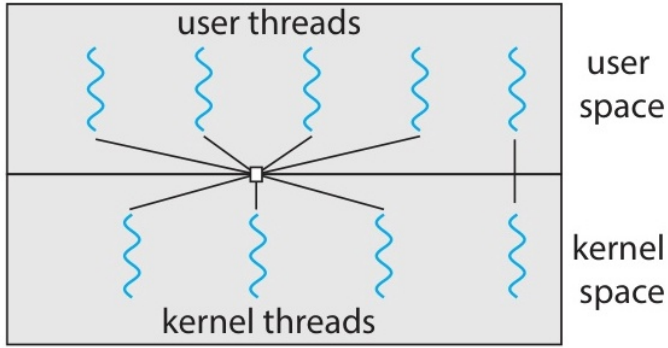
Multiple user threads mapped to an **equal or smaller number of** kernel threads

Advantages:

- Developers can create **as many** user threads as desired
- Kernel threads can execute **in parallel**
- If one thread blocks, the kernel can **schedule another thread**

Disadvantages:

Two-Level Model (Figure 4.10)



Silberschatz, Figure 4.10 – Two-level model

Many-to-Many + **One-to-One** allowed

- Most user threads are managed via many-to-many
- Specific **important threads** can be **directly bound** to a kernel thread
  - Example: threads requiring real-time processing

> In practice, due to implementation complexity, one-to-one became the standard, and many-to-many is used internally by some concurrency libraries

Multithreading Models Summary

Model	Mapping	Advantages	Disadvantages	OS Used
Many-to-One	N:1	Efficient management	Entire process blocks on blocking call, no parallelism	Former Solaris
One-to-One	1:1	High concurrency, parallel execution	Thread count limited (overhead)	**Linux, Windows**
Many-to-Many	N:M	Flexible, parallel + non-blocking	Complex implementation	Some libraries
Two-Level	N:M + 1:1	Flexible + important thread binding	Complex implementation	Former Solaris, IRIX

Key points:

Part 6

Thread Libraries

Silberschatz Ch 4.4

Thread Library Overview

Thread library = provides an **API** for thread creation and management

Two implementation approaches:

1. User-level library (user space)

- All code and data structures reside in user space
- Library function call = **local function call** (not a system call)

2. Kernel-level library (kernel space)

- Code and data structures reside in kernel space
- Library function call = **system call**

Three major thread libraries:

1. **POSIX Pthreads** – user-level or kernel-level
2. **Windows Threads** – kernel-level
3. **Java Threads** – runs on JVM, uses the host OS's library

Asynchronous vs Synchronous Threading

Two main thread creation strategies:

Asynchronous Threading

- Parent thread creates a child thread and they run **independently**
- **Little to no data sharing** between parent and child
- Example: multithreaded web server – each request handled by an independent thread

Synchronous Threading

- Parent thread **waits for all child threads to finish (join)**
- Child threads **pass results** back to the parent upon completion
- **Significant data sharing** between tasks
- Example: parallel summation – the parent combines partial sums from each thread

> All examples in this chapter follow the **synchronous threading** pattern.

Pthreads Overview

POSIX Pthreads = IEEE 1003.1c standard

> It is a **specification**, not an **implementation** OS designers are free to implement it however they choose

Key API functions:

Function	Description
`pthread_create()`	Create a new thread
`pthread_join()`	Wait for a thread to terminate
`pthread_exit()`	Terminate the current thread
`pthread_attr_init()`	Initialize thread attributes

Pthreads Example – Integer Summation (Figure 4.11)

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum;  /* global variable: data shared between threads */
void *runner(void *param);  /* function to be executed by the thread */

int main(int argc, char *argv[])
{
    pthread_t tid;           /* thread identifier */
    pthread_attr_t attr;     /* thread attributes */

    pthread_attr_init(&attr);           /* initialize default attributes */
    pthread_create(&tid, &attr,         /* create the thread */
                   runner, argv[1]);
    pthread_join(tid, NULL);            /* wait for the thread to finish */

    printf("sum = %d\n", sum);
}

void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;
```

Pthreads Example – Code Analysis

Execution flow (input = 5):

1. In `main()`, `pthread_attr_init(&attr)` – set default attributes
2. `pthread_create(&tid, &attr, runner, argv[1])`
   - Create a new thread, begins execution in `runner("5")`
3. `pthread_join(tid, NULL)` – parent **waits** until the child finishes
4. Child thread (`runner`):
   - `upper = 5`, sum = 1+2+3+4+5 = **15**
   - Terminates with `pthread_exit(0)`
5. Parent: `printf("sum = %d\n", sum)` → outputs **"sum = 15"**

Key points:

- `sum` is a **global variable** → shared by all threads
- Without `pthread_join()`, the parent might terminate first
- `runner()` takes a `void *` parameter and returns `void *`

Pthreads – Creating and Joining Multiple Threads

Pattern for joining multiple threads (Figure 4.12):

```
#define NUM_THREADS 10

pthread_t workers[NUM_THREADS];

/* create threads */
for (int i = 0; i < NUM_THREADS; i++)
    pthread_create(&workers[i], NULL, task_func, &args[i]);

/* wait for all threads to finish */
for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

- The creation loop and join loop are **separated**
- All threads run **in parallel**, then are joined sequentially
- A pattern commonly used on multicore systems

pthread_create() in Detail

```c
int pthread_create(
    pthread_t *thread,              /* variable to store the thread ID */
    const pthread_attr_t *attr,     /* thread attributes (NULL = default) */
    void *(*start_routine)(void*), /* function the thread will execute */
    void *arg                       /* argument passed to the function */
);
```

Parameter descriptions:

Parameter	Role
`thread`	Stores the ID of the newly created thread
`attr`	Stack size, scheduling info, etc. (defaults if NULL)
`start_routine`	Function pointer of the form `void *func(void *param)`

Windows Threads Example (Figure 4.13)

```c
#include <windows.h>
#include <stdio.h>

DWORD Sum;  /* shared data (unsigned 32-bit integer) */

DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i ≤ Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param = atoi(argv[1]);

    ThreadHandle = CreateThread(
        NULL,        /* default security attributes */
        0,           /* default stack size */
        Summation,   /* thread function */
        &Param,      /* parameter to thread function */
```

Windows Threads – Comparison with Pthreads

Category	Pthreads	Windows Threads
Header	`<pthread.h>`	`<windows.h>`
Thread creation	`pthread_create()`	`CreateThread()`
Wait for termination	`pthread_join()`	`WaitForSingleObject()`
Wait for multiple	for loop + `pthread_join()`	`WaitForMultipleObjects()`
Thread termination	`pthread_exit()`	`return 0;` or `ExitThread()`
Handle cleanup	Automatic	`CloseHandle()` required

Part 7

Java Threads

Silberschatz Ch 4.4.3

Java Thread Overview

In Java, threads are the **fundamental model of program execution**

- Every Java program runs in at least one thread (the main thread)
- Runs on the **JVM** → uses the host OS's thread library
  - Windows → Windows API
  - Linux/macOS → Pthreads API

Two ways to create threads:

1. **Extend the Thread class** and override `run()`
2. **Implement the Runnable interface** (more common)

> The Runnable approach is recommended – since Java does not support multiple inheritance

Java Thread – Runnable Interface

Method 1: Implement the Runnable interface

```java
class Task implements Runnable {
    public void run() {
        System.out.println("I am a thread.");
    }
}

// Create and start the thread
Thread worker = new Thread(new Task());
worker.start();  // executes run() in a new thread
```

Role of the `start()` method:

1. **Allocates memory and initializes** a new thread in the JVM
2. Calls the `run()` method to make the **thread runnable**

> Note: Calling `run()` directly executes it in the **same thread**! You must call `start()` to create a new thread.

Java Thread – Lambda Expression

Method 2: Lambda expression (Java 8+)

Runnable is a **functional interface** with only one abstract method → Lambdas can be used

```
// Concise expression with Lambda
Runnable task = () -> {
    System.out.println("I am a thread.");
};

Thread worker = new Thread(task);
worker.start();
```

Even more concise:

```
new Thread(() -> {
    System.out.println("I am a thread.");
}).start();
```

> Lambda expressions allow defining thread tasks inline without a separate class definition, making the **code more concise**.

Java Thread – join()

To have the parent thread wait for a child thread to finish, use `join()`:

```
Thread worker = new Thread(new Task());
worker.start();

try {
    worker.join();  // wait until the worker finishes
} catch (InterruptedException ie) {
    // handle interrupt during wait
}

System.out.println("Worker finished!");
```

Pattern for joining multiple threads:

```
Thread[] workers = new Thread[10];
for (int i = 0; i < 10; i++) {
    workers[i] = new Thread(new Task());
    workers[i].start();
}
for (int i = 0; i < 10; i++) {
    try { workers[i].join(); }
```

Java Thread – Comparison with Pthreads/Windows

Category	Pthreads ©	Windows ©	Java
Creation	`pthread_create()`	`CreateThread()`	`new Thread().start()`
Wait	`pthread_join()`	`WaitForSingleObject()`	`thread.join()`
Termination	`pthread_exit()`	`return` / `ExitThread()`	`run()` returns
Shared data	Global variables	Global variables	**Object fields** (no global variables)
Result return	Difficult (`void *`)	Difficult	**Callable/Future**

Java characteristics:

- **No concept of global data**

Part 8

Java Executor Framework

Silberschatz Ch 4.4.3.1

Java Executor Framework Overview

Available since Java 5 in the `java.util.concurrent` package

Executor interface:

```
public interface Executor {
    void execute(Runnable command);
}
```

Difference from the traditional approach:

```
// Traditional: create threads directly
Thread t = new Thread(new Task());
t.start();

// Executor approach: separate thread creation from execution
Executor service = new SomeExecutor();
service.execute(new Task());
```

> **Producer-Consumer model**: produce tasks (Runnable), and threads consume and execute them

Thread Pool Types

Factory methods of the `Executors` class:

Method	Description
`newSingleThreadExecutor()`	Thread pool of size 1 – guarantees sequential execution
`newFixedThreadPool(int n)`	Maintains a fixed pool of n threads
`newCachedThreadPool()`	Creates threads as needed, **reuses** idle threads

```java
// Fixed thread pool example
ExecutorService pool = Executors.newFixedThreadPool(4);

for (int i = 0; i < 100; i++) {
    pool.execute(new Task());  // 100 tasks, processed by 4 threads
}

pool.shutdown();  // shut down the pool after all tasks complete
```

Advantages of Thread Pools

1. Faster response by reusing existing threads

- Saves the cost of creating threads each time
- Already existing threads immediately perform work

2. Limiting the number of threads

- Prevents excessive system resource usage
- Fixed pool: at most n threads run concurrently

3. Separation of task and execution mechanism

- Separates task definition (what to do) from execution strategy (how to execute)
- Enables various strategies such as deferred execution, periodic execution, etc.

> Pool size should be set considering **CPU count, memory, and expected concurrent requests**

Callable and Future – Returning Results

Problem: Runnable's `run()` has no return value (`void`)

Solution: Use `Callable<V>` interface + `Future<V>`

```
import java.util.concurrent.*;

class Summation implements Callable<Integer> {
    private int upper;

    public Summation(int upper) {
        this.upper = upper;
    }

    public Integer call() {
        int sum = 0;
        for (int i = 1; i ≤ upper; i++)
            sum += i;
        return new Integer(sum);
    }
}
```

- The `call()` method of `Callable` can **return a result**
- The returned result is received via a `Future` object

Callable and Future – Complete Example (Figure 4.14)

```
import java.util.concurrent.*;

class Summation implements Callable<Integer> {
    private int upper;
    public Summation(int upper) { this.upper = upper; }

    public Integer call() {
        int sum = 0;
        for (int i = 1; i ≤ upper; i++)
            sum += i;
        return new Integer(sum);
    }
}

public class Driver {
    public static void main(String[] args) {
        int upper = Integer.parseInt(args[0]);
        ExecutorService pool =
            Executors.newSingleThreadExecutor();
        Future<Integer> result =
            pool.submit(new Summation(upper));
```

execute() vs submit()

Method	Parameter	Return Value	Use Case
`execute(Runnable)`	Runnable	None (`void`)	Tasks that do not need a result
`submit(Callable)`	Callable	`Future<V>`	Tasks that **need a result**
`submit(Runnable)`	Runnable	`Future<?>`	Only check completion status

Thread Pool Example (Figure 4.15):

```
import java.util.concurrent.*;

public class ThreadPoolExample {
    public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());
        ExecutorService pool = Executors.newCachedThreadPool();

        for (int i = 0; i < numTasks; i++)
```

JVM and Host OS Relationship

```
Java Application
(Thread, Executor, ... )
```

```
JVM
Java thread API impl.
```

```
Host Operating System
Windows → Windows API
Linux/macOS → Pthreads
```

- The JVM specification does not prescribe **how Java threads are mapped** to the OS
- It varies by implementation:
  - **Windows**: one-to-one model → each Java thread maps to a kernel thread
  - **Linux/macOS**: mapped using the Pthreads API
- As a result, Java threads are also managed by the **OS scheduler**

Part 9

Lab – Pthreads Multithreaded Programming

Lab Overview: Splitting Array Summation Across Multiple Threads

Goal: Divide the summation of a 1000-element array across 4 threads

Applying Data Parallelism:

- Thread 0: sum elements [0] to [249]
- Thread 1: sum elements [250] to [499]
- Thread 2: sum elements [500] to [749]
- Thread 3: sum elements [750] to [999]

```
Array: [  0 ~ 249  |  250 ~ 499  |  500 ~ 749  |  750 ~ 999  ]
           ↓             ↓             ↓             ↓
       Thread 0      Thread 1      Thread 2      Thread 3
       partial[0]    partial[1]    partial[2]    partial[3]
           ↘             ↓             ↓            ↙
                     Total Sum
```

Lab: Thread Function Implementation

```c
#include <pthread.h>
#include <stdio.h>

#define NUM_THREADS 4
#define ARRAY_SIZE  1000

int array[ARRAY_SIZE];
int partial_sum[NUM_THREADS];

void *sum_array(void *arg)
{
    int id    = *(int *)arg;
    int chunk = ARRAY_SIZE / NUM_THREADS;
    int start = id * chunk;
    int end   = start + chunk;

    partial_sum[id] = 0;
    for (int i = start; i < end; i++)
        partial_sum[id] += array[i];

    printf("Thread %d: partial_sum = %d\n",
           id, partial_sum[id]);
    pthread_exit(NULL);
}
```

Lab: Main Function

```c
int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    /* Initialize array: array[i] = i + 1 */
    for (int i = 0; i < ARRAY_SIZE; i++)
        array[i] = i + 1;

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL,
                       sum_array, &thread_ids[i]);
    }

    /* Wait for all threads and accumulate results */
    int total = 0;
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
        total += partial_sum[i];
    }

    printf("Total sum = %d\n", total);
```

Lab: Code Analysis

Execution flow:

1. `array[i] = i + 1` → [1, 2, 3, …, 1000]
2. 4 threads are created, each responsible for 250 elements
3. Thread 0: 1+2+…+250 = 31375
4. Thread 1: 251+252+…+500 = 93875
5. Thread 2: 501+502+…+750 = 156375
6. Thread 3: 751+752+…+1000 = 218875
7. `pthread_join()` waits for all threads to complete
8. Total = 31375 + 93875 + 156375 + 218875 = **500500**

Compilation and execution:

```
gcc -pthread lab_sum.c -o lab_sum
./lab_sum
```

> The `-pthread` flag is required (links the Pthreads library)

Lab: Why the thread_ids Array Is Necessary

Incorrect code (a common mistake):

```
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_create(&threads[i], NULL,
                   sum_array, &i);  /* Dangerous! */
}
```

The problem:

- All threads share the **same address of variable** `i`
- By the time a thread executes, the value of `i` may have already changed
- Example: all 4 threads might read `id = 4`

Correct code:

```
int thread_ids[NUM_THREADS];
for (int i = 0; i < NUM_THREADS; i++) {
    thread_ids[i] = i;  /* separate variable for each thread */
    pthread_create(&threads[i], NULL,
                   sum_array, &thread_ids[i]);
}
```

Lab: Observing a Race Condition

What if all threads share **a single global variable** instead of `partial_sum[id]`?

```
int global_sum = 0;  /* shared by all threads */

void *sum_array_bad(void *arg) {
    int id    = *(int *)arg;
    int start = id * (ARRAY_SIZE / NUM_THREADS);
    int end   = start + (ARRAY_SIZE / NUM_THREADS);

    for (int i = start; i < end; i++)
        global_sum += array[i];  /* Race condition! */

    pthread_exit(NULL);
}
```

Race condition occurs:

- `global_sum += array[i]` is a read-modify-write 3-step operation
- When multiple threads execute it simultaneously, **values can be lost**
- Running it multiple times may produce **different results**

> Synchronization is covered in detail in Ch 6 and 7

Lab: Integer Summation with Pthreads (Textbook Example Variant)

Summing integers received as a command-line argument:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum = 0;

void *runner(void *param)
{
    int upper = atoi(param);
    for (int i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <integer>\n", argv[0]);
        return 1;
    }
```

Lab Key Takeaways

Basic Pthreads pattern:

1. `pthread_attr_init()` – initialize attributes (defaults are fine)
2. `pthread_create()` – create a thread (function pointer + argument)
3. Perform work in the thread function (share results via global variables)
4. `pthread_join()` – wait for termination
5. Collect and print results

Important notes:

- Use **separate variables** when passing arguments to thread functions (to prevent race conditions)
- **Synchronization** is needed when accessing shared variables (avoided in this lab by using separate indices)
- The `-pthread` flag is required at compile time

Part 10

Summary and Next Week Preview

Summary

What we covered this week:

- **Concept of threads**: unit of execution within a process; shares code/data/files; PC/registers/stack are independent
- **Benefits of threads**: Responsiveness, Resource Sharing, Economy, Scalability
- **Multicore Programming**: Concurrency vs Parallelism, 5 challenges
- **Amdahl's Law**: serial fraction is the bottleneck – speedup <= 1/S
- **Data Parallelism vs Task Parallelism**: data partitioning vs task partitioning
- **Multithreading Models**:
  - Many-to-One (rarely used), **One-to-One** (Linux, Windows), Many-to-Many (complex)
- **Thread Libraries**:
  - **Pthreads** – `pthread_create()`, `pthread_join()`, `pthread_exit()`
  - **Windows** – `CreateThread()`, `WaitForSingleObject()`
  - **Java** – `Runnable`, `Thread.start()`, `Thread.join()`, Lambda
- **Java Executor Framework**: Thread pool, `Callable`, `Future`

Next Week Preview

Week 5 – Threads and Concurrency 2 (Ch 4 second half)

- Implicit Threading
  - Thread Pool
  - Fork-Join
  - OpenMP
  - Grand Central Dispatch (GCD)
- Threading Issues
  - fork() / exec() in multithreaded programs
  - Signal handling
  - Thread cancellation
  - Thread-Local Storage (TLS)
- OS-specific Thread Implementations (Linux, Windows)

Textbook: Ch 4 (Sections 4.5 – 4.8)